

Laporan Modul 6: Model dan Laravel Eloquent

Mata Kuliah: Workshop Web Lanjut

Nama: Faizul Abrar

NIM: 2024573010103

Kelas: TI-2C

Abstrak

Pada praktikum ini mahasiswa mempelajari konsep dasar **Model** dan penerapan **Eloquent ORM (Object Relational Mapping)** dalam framework Laravel.

Eloquent merupakan fitur bawaan Laravel yang digunakan untuk berinteraksi dengan database menggunakan sintaks berorientasi objek, tanpa harus menulis query SQL secara manual.

Tujuan dari praktikum ini adalah memahami bagaimana model bekerja, cara membuat relasi antar tabel seperti *One to Many* dan *Many to Many*, serta bagaimana melakukan operasi CRUD menggunakan Eloquent.

1. Dasar Teori

Model dalam Laravel berfungsi sebagai representasi dari tabel di dalam database.

Dengan menggunakan Eloquent, pengembang dapat mengakses, menyimpan, dan memanipulasi data hanya dengan memanggil metode pada model, bukan melalui query SQL mentah.

Setiap model umumnya berhubungan langsung dengan satu tabel database, dan setiap instance model merepresentasikan satu baris data.

Beberapa konsep penting dalam Eloquent antara lain:

- **Eloquent Model:** Digunakan untuk mendefinisikan tabel dan interaksi dengan data.
- **Migration:** Berfungsi untuk membuat dan mengatur struktur tabel database.
- **Seeder:** Digunakan untuk mengisi data awal ke dalam tabel.
- **Relasi:** Menghubungkan antar model, seperti:
 - *One to One*
 - *One to Many*
 - *Many to Many*
- **Query Builder:** Memudahkan proses pencarian dan manipulasi data dengan metode berorientasi objek.

Dengan Eloquent, proses pengembangan aplikasi berbasis data menjadi lebih cepat, aman, dan mudah dibaca.

2. Langkah-Langkah Praktikum

Praktikum 1: Membuat Model dan Migration

Langkah 1: Membuat Proyek Baru

```
laravel new modul6-eloquent
cd modul6-eloquent
code .
```

Langkah 2: Konfigurasi Database

Buka file `.env` dan sesuaikan konfigurasi dengan database lokal:

```
DB_DATABASE=modul6
DB_USERNAME=root
DB_PASSWORD=
```

Langkah 3: Membuat Model dan Migration

Buat model `Post` sekaligus dengan migrasinya:

```
php artisan make:model Post -m
```

Edit file `database/migrations/xxxx_xx_xx_create_posts_table.php`:

```
public function up()
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->text('content');
        $table->timestamps();
    });
}
```

Jalankan migrasi:

```
php artisan migrate
```

Praktikum 2: Menggunakan Seeder untuk Menambah Data Awal

Buat seeder baru:

```
php artisan make:seeder PostSeeder
```

Edit file `database/seeder/PostSeeder.php`:

```
namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Post;

class PostSeeder extends Seeder
{
    public function run()
    {
        Post::create([
            'title' => 'Belajar Laravel Eloquent',
            'content' => 'Eloquent adalah ORM bawaan Laravel yang sangat
powerful.'
        ]);

        Post::create([
            'title' => 'Model dan Migration',
            'content' => 'Model digunakan untuk berinteraksi dengan tabel
database.'
        ]);
    }
}
```

Tambahkan seeder di `DatabaseSeeder.php`:

```
public function run(): void
{
    $this->call(PostSeeder::class);
}
```

Jalankan seeder:

```
php artisan db:seed
```

Praktikum 3: Membuat Relasi One to Many

Langkah 1: Membuat Model `Comment`

```
php artisan make:model Comment -m
```

Edit file `database/migrations/xxxx_xx_xx_create_comments_table.php`:

```
public function up()
{
    Schema::create('comments', function (Blueprint $table) {
        $table->id();
        $table->foreignId('post_id')->constrained()->onDelete('cascade');
        $table->string('author');
        $table->text('message');
        $table->timestamps();
    });
}
```

Jalankan migrasi:

```
php artisan migrate
```

Langkah 2: Menentukan Relasi antar Model

Edit `app/Models/Post.php`:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    use HasFactory;

    protected $fillable = ['title', 'content'];

    public function comments()
    {
        return $this->hasMany(Comment::class);
    }
}
```

Edit `app/Models/Comment.php`:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    use HasFactory;
```

```
protected $fillable = ['post_id', 'author', 'message'];

public function post()
{
    return $this->belongsTo(Post::class);
}
}
```

Langkah 3: Menguji Relasi di Tinker

```
php artisan tinker
```

Kemudian jalankan:

```
$post = App\Models\Post::first();
$post->comments()->create(['author' => 'Faizul', 'message' => 'Laravel sangat
membantu!']);
$post->comments;
```

Praktikum 4: Membuat Relasi Many to Many

Langkah 1: Membuat Model Tag

```
php artisan make:model Tag -m
```

Edit migrasi `create_tags_table.php`:

```
public function up()
{
    Schema::create('tags', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->timestamps();
    });
}
```

Buat tabel pivot `post_tag`:

```
php artisan make:migration create_post_tag_table
```

Isi migrasinya:

```
public function up()
{
    Schema::create('post_tag', function (Blueprint $table) {
        $table->id();
        $table->foreignId('post_id')->constrained()->onDelete('cascade');
        $table->foreignId('tag_id')->constrained()->onDelete('cascade');
    });
}
```

Jalankan semua migrasi:

```
php artisan migrate
```

Langkah 2: Definisikan Relasi di Model

Edit `app/Models/Post.php`:

```
public function tags()
{
    return $this->belongsToMany(Tag::class);
}
```

Edit `app/Models/Tag.php`:

```
public function posts()
{
    return $this->belongsToMany(Post::class);
}
```

Langkah 3: Uji Relasi Many to Many

```
php artisan tinker
```

Lalu jalankan:

```
$tag = App\Models\Tag::create(['name' => 'Laravel']);
$post = App\Models\Post::first();
$post->tags()->attach($tag->id);
$post->tags;
```

3. Hasil dan Pembahasan

Dari hasil praktikum, mahasiswa berhasil membuat model dan relasi antar tabel menggunakan Eloquent ORM. Data dapat dimasukkan, dihubungkan, serta ditampilkan melalui relasi *One to Many* dan *Many to Many*. Eloquent mempermudah proses CRUD dengan sintaks yang sederhana tanpa menulis query SQL secara langsung.

Seluruh proses menunjukkan bagaimana Laravel mengotomatisasi manajemen relasi database, serta menjaga konsistensi data dengan fitur *cascade delete*.

Penggunaan Tinker juga sangat membantu dalam melakukan pengujian relasi secara cepat dan interaktif.

4. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa Eloquent ORM adalah komponen penting dalam framework Laravel untuk mengelola data dan relasi antar tabel secara efisien.

Dengan fitur seperti *Model*, *Migration*, dan *Relationship*, pengembang dapat bekerja dengan database secara lebih terstruktur, aman, dan mudah dipelihara.

Eloquent juga mendukung berbagai jenis relasi yang umum digunakan dalam aplikasi modern.

5. Referensi

- <https://hackmd.io/@mohdrzu/BJShg5tx1g>
 - <https://laravel.com/docs/12.x/eloquent>
 - ChatGPT (penjelasan tambahan konsep)
-